

## adaptive\_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Adaptive Rag
4 # =====
5 # Description:
6 #   Получить стандартный путь к локальному хранилищу в %APPDAT
  A%.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.src.adaptive_rag impo
  rt AdaptiveLogRAG
11 #
12 #       service = AdaptiveLogRAG()
13 #
14 # File: adaptive_rag.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.src
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 05:03:00
20 # =====
21
22 from __future__ import annotations
23 """Адаптивный движок создания и поиска RAG-документов с интегр
  ацией в базу знаний WikiLLM."""
24
25 import collections
26 import datetime
27 import hashlib
28 import json
29 import math
```

```

30 import os
31 import re
32 from pathlib import Path
33 from typing import Any, Dict, List, Optional, Tuple
34
35 from logger import logger
36 from .models import DataProfileReport, IngestionDecision, Inge
    stionStrategy
37
38 from apps.windows.wikillm.models import (
39     ArtifactType,
40     Claim,
41     DiagnosticKnowledge,
42     KnowledgeEntity,
43     KnowledgeSource,
44     ResolutionAction,
45 )
46 from apps.windows.wikillm.storage import WikiStorage
47
48
49 def get_default_storage_dir() -> Path:
50     """Получить стандартный путь к локальному хранилищу в %APP
    DATA%."""
51     appdata_env = os.environ.get('APPDATA')
52     if appdata_env:
53         base_dir = Path(appdata_env) / 'AI-Breadboard' / 'apps
    ' / 'windows' / 'log_intelligence'
54     else:
55         user_profile = os.environ.get('USERPROFILE') or os.env
    iron.get('HOME')
56         if user_profile:
57             base_dir = Path(user_profile) / 'AppData' / 'Roami
    ng' / 'AI-Breadboard' / 'apps' / 'windows' / 'log_intelligence
    '
58         else:
59             base_dir = Path('.').resolve() / 'data' / 'system_
    logs_rag'
60     base_dir.mkdir(parents=True, exist_ok=True)

```

```

61 |     return base_dir
62 |
63 |
64 | class AdaptiveLogRAG:
65 |     """Адаптивный движок создания и поиска RAG-документов с по
        ддержкой хранилища WikiLLM."""
66 |
67 |     def __init__(self, storage_dir: Optional[Path] = None, max
        _chunks: int = 1000) -> None:
68 |         """Инициализировать адаптивный RAG-индекс в локальном
        хранилище.
69 |
70 |         Args:
71 |             storage_dir: Путь к директории хранения.
72 |             max_chunks: Максимальное количество чанков в опера
        тивной памяти.
73 |         """
74 |         self.storage_dir = Path(storage_dir).resolve() if stor
        age_dir else get_default_storage_dir()
75 |         self.storage_dir.mkdir(parents=True, exist_ok=True)
76 |         self.max_chunks = max_chunks
77 |         self.chunks: List[Dict[str, Any]] = []
78 |
79 |         # Интеграция с движком WikiStorage SQLite
80 |         wiki_db_path = self.storage_dir / 'wikillm.db'
81 |         self.wiki_storage = WikiStorage(db_path=wiki_db_path)
82 |
83 |         self._load()
84 |
85 |     def _get_chunks_file(self) -> Path:
86 |         return self.storage_dir / 'adaptive_log_rag_chunks.jso
        n'
87 |
88 |     def _get_history_file(self) -> Path:
89 |         return self.storage_dir / 'profiling_history.json'
90 |
91 |     def _load(self) -> None:
92 |         c_file = self._get_chunks_file()

```

```

93         if c_file.exists():
94             try:
95                 with open(c_file, 'r', encoding='utf-8') as f:
96                     self.chunks = json.load(f)
97             except Exception as e:
98                 logger.error(f'[AdaptiveLogRAG] Ошибка загрузк
и чанков: {e}')
99                 self.chunks = []
100
101     def save(self) -> None:
102         c_file = self._get_chunks_file()
103         try:
104             with open(c_file, 'w', encoding='utf-8') as f:
105                 json.dump(self.chunks, f, ensure_ascii=False,
indent=2)
106         except Exception as e:
107             logger.error(f'[AdaptiveLogRAG] Ошибка сохранения
чанков: {e}')
108
109     def ingest(self, profile: DataProfileReport, decision: Ing
estionDecision) -> int:
110         """Сгенерировать целевые RAG-документы и сохранить их
в базу знаний WikiLLM.
111
112         Args:
113             profile (DataProfileReport): Аналитический срез да
нных.
114             decision (IngestionDecision): Решение и стратегия
шлюза.
115
116         Returns:
117             int: Количество добавленных / обновленных чанков.
118         """
119         now_str = profile.generated_at or datetime.datetime.no
w().strftime('%Y-%m-%d %H:%M:%S')
120         generated_chunks: List[Dict[str, Any]] = []
121
122         snap_id = f'snapshot_{profile.channel}_{now_str[:13]}'

```

```

123         snap_text = (
124             f'Аналитический снимок здоровья канала {profile.ch
annel}.\n'
125             f'Метка времени: {now_str}\n'
126             f'Индекс здоровья (Health Score): {profile.health_
score} / 100\n'
127             f'Всего событий: {profile.total_events}, Уникальны
х шаблонов: {profile.unique_templates_count}\n'
128             f'Избыточность/дублирование: {profile.redundancy_r
atio_pct}%\n'
129             f'Ошибок: {profile.error_count + profile.critical_
count}, Предупреждений: {profile.warning_count}\n'
130             f'Выбранная стратегия: {decision.strategy.value}\n
',
131             f'Обоснование: {decision.rationale}'
132         )
133         generated_chunks.append({
134             'id': snap_id,
135             'type': 'health_snapshot',
136             'channel': profile.channel,
137             'title': f'Снимок здоровья {profile.channel} ({pro
file.health_score}%)',
138             'text': snap_text,
139             'timestamp': now_str,
140             'meta': {'strategy': decision.strategy.value, 'hea
lth_score': profile.health_score},
141         })
142
143         # Регистрация сущности снимка в WikiStorage
144         snap_entity = KnowledgeEntity(
145             canonical_key=f"snapshot:{profile.channel}:{now_st
r[:13]}",
146             entity_type=ArtifactType.INCIDENT,
147             name=f"Снимок здоровья {profile.channel} ({profile
.health_score}%)",
148             summary=snap_text,
149             severity="info" if profile.health_score >= 80 else
("warning" if profile.health_score >= 50 else "error"),

```

```

150         confidence=1.0,
151         tags=[profile.channel.lower(), "health_snapshot",
152         f"score_{int(profile.health_score)}"],
153     )
154     self.wiki_storage.save_entity(snap_entity)
155     self.wiki_storage.record_observation(snap_entity.canonical_key)
156
157     if decision.strategy == IngestionStrategy.INCIDENT_FOCUSED:
158         for idx, b in enumerate(profile.bursts):
159             b_id = f"burst_{profile.channel}_{b.time_window.replace(':', '_')}_{idx}"
160             b_text = (
161                 f'Инцидентный каскад событий в канале {profile.channel}.\n'
162                 f'Окно всплеска: {b.time_window}\n'
163                 f'Событий в пике: {b.event_count}\n'
164                 f'Доминирующий уровень: {b.dominant_level}\n'
165                 f'Ключевой источник: {b.dominant_provider}\n'
166                 f'Сводка: {b.summary}'
167             )
168             generated_chunks.append({
169                 'id': b_id,
170                 'type': 'incident_burst',
171                 'channel': profile.channel,
172                 'title': f'Инцидентный всплеск [{b.dominant_provider} ({b.dominant_level})]',
173                 'text': b_text,
174                 'timestamp': now_str,
175                 'meta': {'window': b.time_window, 'provider': b.dominant_provider},
176             })
177
178             burst_entity = KnowledgeEntity(
179                 canonical_key=f"burst:{profile.channel}:{b"

```

```

    .dominant_provider}:{idx}",
179         entity_type=ArtifactType.INCIDENT,
180         name=f"Инцидентный всплеск [{b.dominant_pr
    ovider} ({b.dominant_level})]",
181         summary=b_text,
182         severity="error" if b.dominant_level in ("
    Error", "Critical") else "warning",
183         confidence=0.95,
184         diagnostic_info=DiagnosticKnowledge(
185             symptoms=[b.summary],
186             possible_causes=[f"Всплеск активности
    провайдера {b.dominant_provider} в окне {b.time_window}"],
187             ),
188             tags=[profile.channel.lower(), b.dominant_
    provider.lower(), "incident_burst"],
189             )
190             self.wiki_storage.save_entity(burst_entity)
191             self.wiki_storage.record_observation(burst_ent
    ity.canonical_key)
192
193             if decision.strategy in (IngestionStrategy.NOVELTY_SIG
    NATURE, IngestionStrategy.INCIDENT_FOCUSED):
194                 for sig in profile.novel_signatures:
195                     sig_id = f"sig_{profile.channel}_{sig['provide
    r']}_{sig['event_id']}"
196                     sig_text = (
197                         f"Новая сигнатура журнала {profile.channel
    }. \n"
198                         f"Поставщик: {sig['provider']}, Event ID:
    {sig['event_id']} \n"
199                         f"Уровень: {sig['level']}, Встретилось раз
    : {sig['count']} \n"
200                         f"Образец: {sig['sample']}"
201                     )
202                     generated_chunks.append({
203                         'id': sig_id,
204                         'type': 'novel_signature',
205                         'channel': profile.channel,

```

```

206         'title': f"Сигнатура {sig['provider']} (ID
    {sig['event_id']})",
207         'text': sig_text,
208         'timestamp': now_str,
209         'meta': {'provider': sig['provider'], 'event_id': sig['event_id']},
210     })
211
212     sig_entity = KnowledgeEntity(
213         canonical_key=f"windows_event:{sig['provider']}:{sig['event_id']}",
214         entity_type=ArtifactType.WINDOWS_EVENT,
215         name=f"Сигнатура {sig['provider']} (ID {sig['event_id']})",
216         summary=sig_text,
217         fingerprint_pattern=sig.get('sample', ''),
218         severity="error" if sig['level'] in ("Error", "Critical") else "warning",
219         confidence=0.9,
220         tags=[profile.channel.lower(), str(sig['provider']).lower(), f"event_{sig['event_id']}"],
221     )
222     self.wiki_storage.save_entity(sig_entity)
223     self.wiki_storage.record_observation(sig_entity.canonical_key)
224
225     if decision.strategy == IngestionStrategy.NOISE_MASKED:
226         for p in profile.top_patterns[:3]:
227             p_id = f"pattern_{profile.channel}_{p['provider']}_{p['event_id']}_{hashlib.md5(p['template'].encode('utf-8')).hexdigest()[:6]}"
228             p_text = (
229                 f"Сжатый дайджест фонового сервиса {p['provider']}. \n"
230                 f"Event ID: {p['event_id']}, Уровень: {p['level']} \n"
231                 f"Количество повторений: {p['count']} шт. \n"

```

```

n"
232         f"Шаблон сообщения: {p['template']}"
233     )
234     generated_chunks.append({
235         'id': p_id,
236         'type': 'noise_digest',
237         'channel': profile.channel,
238         'title': f"Фоновый шаблон [{p['provider']}
x{p['count']}]\"",
239         'text': p_text,
240         'timestamp': now_str,
241         'meta': {'provider': p['provider'], 'count
': p['count']},
242     })
243
244     pat_entity = KnowledgeEntity(
245         canonical_key=f"pattern:{profile.channel}:
{p['provider']}:{p['event_id']}",
246         entity_type=ArtifactType.SYMPTOM,
247         name=f"Фоновый шаблон [{p['provider']} x{p
['count']}]\"",
248         summary=p_text,
249         fingerprint_pattern=p.get('template', ''),
250         severity="info",
251         confidence=1.0,
252         tags=[profile.channel.lower(), str(p['prov
ider']).lower(), "noise_digest"],
253     )
254     self.wiki_storage.save_entity(pat_entity)
255     self.wiki_storage.record_observation(pat_entit
y.canonical_key)
256
257     doc_dict: Dict[str, Dict[str, Any]] = {c['id']: c for
c in self.chunks}
258     for chunk in generated_chunks:
259         doc_dict[chunk['id']] = chunk
260     updated = list(doc_dict.values())
261     updated.sort(key=lambda x: x.get('timestamp', ''), rev

```

```

    erse=True)
262         self.chunks = updated[:self.max_chunks]
263         self.save()
264         return len(generated_chunks)
265
266     def search(self, query: str, top_k: int = 5, channel: str
= '') -> List[Dict[str, Any]]:
267         """Быстрый гибридный поиск по адаптивным чанкам RAG и
SQLite FTS5 базе WikiLLM."""
268         if not query.strip():
269             return []
270
271         results_by_id: Dict[str, Dict[str, Any]] = {}
272
273         # 1. Полнотекстовый поиск FTS5 через WikiStorage
274         try:
275             fts_entities = self.wiki_storage.search_fts(query,
limit=top_k)
276             for entity in fts_entities:
277                 key = entity.canonical_key
278                 results_by_id[key] = {
279                     'id': key,
280                     'type': entity.entity_type.value,
281                     'channel': channel or 'System',
282                     'title': entity.name,
283                     'text': entity.summary,
284                     'timestamp': entity.updated_at,
285                     'meta': {'severity': entity.severity, 'con
fidence': entity.confidence, 'source': 'wikillm_fts'},
286                     'relevance_score': 2.0,
287                 }
288         except Exception as fts_err:
289             logger.debug(f"[AdaptiveLogRAG] Ошибка FTS поиска
в WikiStorage: {fts_err}")
290
291         # 2. Локальный поиск по ключевым словам и синонимам
292         if self.chunks:
293             candidates = [c for c in self.chunks if not channe

```

```

    l or c.get('channel', '').lower() == channel.lower()]
294         if not candidates:
295             candidates = self.chunks
296
297         cleaned = re.sub(r'^\w\s-', ' ', query.lower())
298         tokens = [t.strip() for t in cleaned.split() if le
n(t.strip()) > 1]
299         if tokens:
300             synonyms = {
301                 'сеть': ['network', 'nic', 'realtek', 'vms
witch', 'disconnect', 'адаптер'],
302                 'сбой': ['error', 'critical', 'crash', 'fa
ult', 'инцидент', 'ошибка'],
303                 'здоровье': ['health', 'snapshot', 'стабил
ьность', 'score'],
304                 'служба': ['service', 'scm', '7040', '7036
'],
305             }
306             expanded = list(tokens)
307             for t in tokens:
308                 for k, syn_list in synonyms.items():
309                     if t.startswith(k[:3]) or k.startswith
(t[:3]):
310                         expanded.extend(syn_list)
311
312             for c in candidates:
313                 c_id = c.get('id', '')
314                 c_text = f"{c.get('title', '')} {c.get('te
xt', '')}".lower()
315                 score = 0.0
316                 for tok in expanded:
317                     if tok in c_text:
318                         score += 1.0
319                 if score > 0:
320                     if c_id in results_by_id:
321                         results_by_id[c_id]['relevance_sco
re'] += score
322                 else:

```

```
323 |             res = dict(c)
324 |             res['relevance_score'] = score
325 |             results_by_id[c_id] = res
326 |
327 |         final_results = list(results_by_id.values())
328 |         final_results.sort(key=lambda x: x.get('relevance_score', 0.0), reverse=True)
329 |         return final_results[:top_k]
```

data\_researcher.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Data Researcher
4 # =====
5 # Description:
6 #   Аналитический движок статистического профилирования и EDA
  системных журналов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.src.data_researcher i
  mport LogDataResearcher
11 #
12 #       service = LogDataResearcher()
13 #
14 # File: data_researcher.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.src
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Аналитический движок статистического профилирования и EDA с
  системных журналов."""
24
25 import collections
26 import datetime
27 import math
28 import re
29 from typing import Any, Dict, List, Optional, Set
```

```

30 from logger import logger
31 from .models import DataProfileReport, LogEntry, LogSeverity,
   TimeBurst
32
33 class LogDataResearcher:
34     """Аналитический движок статистического профилирования и E
   DA системных журналов."""
35
36     def __init__(self, known_signatures_catalog: Optional[Set[
   str]]=None) -> None:
37         """Инициализация профайлера с каталогом известных сигн
   атур."""
38         self.known_signatures = known_signatures_catalog or {'
   Service Control Manager:7036', 'Service Control Manager:7040',
   'Microsoft-Windows-Kernel-PnP:566', 'Microsoft-Windows-Kernel
   -PnP:902', 'Microsoft-Windows-Kernel-PnP:903', 'Microsoft-Wind
   ows-DistributedCOM:10016'}
39
40     @staticmethod
41     def extract_template(message: str) -> str:
42         """Извлечь обобщенный шаблон из сообщения путем маскир
   ования динамических сущностей."""
43         if not message:
44             return ''
45         text = re.sub('[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-
   F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}', '<GUID>', message)
46         text = re.sub('0x[0-9a-fA-F]+', '<HEX>', text)
47         text = re.sub('\\b\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d
   {1,3}\\b', '<IP>', text)
48         text = re.sub('[a-zA-Z]:\\\\\\\\[^\s,;]+' , '<PATH>', text
   )
49         text = re.sub('\\b\\d+\\b', '<NUM>', text)
50         return re.sub '\\s+', ' ', text).strip()
51
52     def profile_data(self, entries: List[LogEntry], channel: s
   tr='System') -> DataProfileReport:
53         """Выполнить полный статистический срез и профилирован
   ие массива логов (EDA).

```

```

54 |
55 |     Args:
56 |         entries (List[LogEntry]): Входной массив записей.
57 |         channel (str): Название канала.
58 |
59 |     Returns:
60 |         DataProfileReport: Структурированный аналитический
61 |         профиль.
62 |         """
63 |         now_str = datetime.datetime.now().strftime('%Y-%m-%d %
64 |         H:%M:%S')
65 |         total = len(entries)
66 |         if total == 0:
67 |             return DataProfileReport(channel=channel, total_ev
68 |             ents=0, unique_templates_count=0, redundancy_ratio_pct=0.0, he
69 |             alth_score=100.0, generated_at=now_str)
70 |         crit_count = 0
71 |         err_count = 0
72 |         warn_count = 0
73 |         info_count = 0
74 |         template_map: Dict[tuple, Dict[str, Any]] = collection
75 |         s.defaultdict(lambda: {'count': 0, 'first_seen': None, 'last_s
76 |         een': None, 'provider': '', 'event_id': 0, 'level': 'Informati
77 |         on', 'template': '', 'sample': ''})
78 |         minute_buckets: Dict[str, Dict[str, Any]] = collection
79 |         s.defaultdict(lambda: {'count': 0, 'levels': collections.Count
80 |         er(), 'providers': collections.Counter()})
81 |         provider_counts = collections.Counter()
82 |         for e in entries:
83 |             lvl = (e.level or 'Information').capitalize()
84 |             lvl_lower = lvl.lower()
85 |             if 'crit' in lvl_lower:
86 |                 crit_count += 1
87 |             elif 'err' in lvl_lower:
88 |                 err_count += 1
89 |             elif 'warn' in lvl_lower:
90 |                 warn_count += 1
91 |             else:

```

```

83         info_count += 1
84         prov = e.provider or e.source or 'Unknown'
85         provider_counts[prov] += 1
86         tpl = self.extract_template(e.message)
87         key = (prov, e.event_id, lvl, tpl[:100])
88         t_item = template_map[key]
89         t_item['count'] += 1
90         t_item['provider'] = prov
91         t_item['event_id'] = e.event_id
92         t_item['level'] = lvl
93         t_item['template'] = tpl
94         if not t_item['sample'] and e.message:
95             t_item['sample'] = e.message
96         if not t_item['first_seen'] or (e.timestamp and e.
timestamp < t_item['first_seen']):
97             t_item['first_seen'] = e.timestamp
98         if not t_item['last_seen'] or (e.timestamp and e.t
imestamp > t_item['last_seen']):
99             t_item['last_seen'] = e.timestamp
100         if len(e.timestamp) >= 16:
101             min_key = e.timestamp[:16]
102             mb = minute_buckets[min_key]
103             mb['count'] += 1
104             mb['levels'][lvl] += 1
105             mb['providers'][prov] += 1
106         unique_templates = len(template_map)
107         redundancy_pct = round((1.0 - unique_templates / total
) * 100.0, 2)
108         penalty = (crit_count * 15.0 + err_count * 8.0 + warn_
count * 1.5) / max(total, 1) * 100.0
109         health_score = max(0.0, min(100.0, round(100.0 - penal
ty, 1)))
110         dom_noise = provider_counts.most_common(1)[0][0] if pr
ovider_counts else ''
111         bursts: List[TimeBurst] = []
112         if minute_buckets:
113             avg_per_min = total / len(minute_buckets)
114             burst_threshold = max(5, int(avg_per_min * 2.5))

```

```

115         for m_time, m_data in sorted(minute_buckets.items(
116         )):
117             if m_data['count'] >= burst_threshold or m_data['levels']['Error'] > 0 or m_data['levels']['Critical'] > 0:
118                 dom_lvl = m_data['levels'].most_common(1)[0][0] if m_data['levels'] else 'Info'
119                 dom_prov = m_data['providers'].most_common(1)[0][0] if m_data['providers'] else 'System'
120                 bursts.append(TimeBurst(time_window=m_time, event_count=m_data['count'], dominant_level=dom_lvl, dominant_provider=dom_prov, summary=f"Всплеск {m_data['count']} событий в {m_time} ({dom_lvl} от {dom_prov})"))
121                 critical_incidents: List[Dict[str, Any]] = []
122                 novel_sigs: List[Dict[str, Any]] = []
123                 for (prov, eid, lvl, _), t_data in template_map.items():
124                     sig_key = f'{prov}:{eid}'
125                     if lvl.lower() in ('critical', 'error'):
126                         critical_incidents.append({'provider': prov, 'event_id': eid, 'level': lvl, 'count': t_data['count'], 'sample': t_data['sample'][:120], 'template': t_data.get('template', '')})
127                     if sig_key not in self.known_signatures and eid > 0:
128                         novel_sigs.append({'provider': prov, 'event_id': eid, 'level': lvl, 'count': t_data['count'], 'sample': t_data['sample'][:120]})
129                 sorted_patterns = sorted(template_map.values(), key=lambda x: x['count'], reverse=True)[:10]
130                 return DataProfileReport(channel=channel, total_events=total, unique_templates_count=unique_templates, redundancy_ratio_pct=redundancy_pct, health_score=health_score, critical_count=crit_count, error_count=err_count, warning_count=warn_count, info_count=info_count, dominant_noise_provider=dom_noise, bursts=bursts[:5], critical_incidents=critical_incidents[:10], novel_signatures=novel_sigs[:5], top_patterns=sorted_patterns, generated_at=now_str)

```

## decision\_gate.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Decision Gate
4 # =====
5 # Description:
6 #   Интеллектуальный шлюз принятия решений перед созданием RAG
  .
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.src.decision_gate imp
  ort DecisionGate
11 #
12 #       service = DecisionGate()
13 #
14 # File: decision_gate.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.src
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Интеллектуальный шлюз принятия решений перед созданием RAG.
  """
24
25 from typing import List
26 from .models import DataProfileReport, IngestionDecision, Inge
  stionStrategy
27
28 class DecisionGate:
```

```

29 |     """Интеллектуальный шлюз принятия решений перед созданием
    | RAG."""
30 |
31 |     def evaluate(self, profile: DataProfileReport) -> IngestionDecision:
32 |         """Оценить аналитический профиль и выбрать наилучшую стратегию RAG.
33 |
34 |         Args:
35 |             profile (DataProfileReport): Профиль данных от Data Researcher.
36 |
37 |         Returns:
38 |             IngestionDecision: Решение с выбранной стратегией и планом действий.
39 |         """
40 |         if profile.critical_count > 0 or profile.error_count > 0 or profile.health_score < 75.0:
41 |             windows = [b.time_window for b in profile.bursts if b.dominant_level in ('Critical', 'Error', 'Warning')]
42 |             return IngestionDecision(strategy=IngestionStrategy.INCIDENT_FOCUSED, rationale=f'Обнаружен инцидентный сбой: {profile.critical_count} крит., {profile.error_count} ошибок. Health Score: {profile.health_score}%. Векторизуются только цепочки событий вокруг сбоя.', chunks_to_generate=min(12, max(2, len(profile.bursts) + len(profile.novel_signatures))), target_time_windows=windows, skip_providers=[profile.dominant_noise_provider] if profile.redundancy_ratio_pct > 80 else [], recommended_llm_action='Запустить AI-диагностику причин падения и цепочки каскада.')
43 |         if profile.novel_signatures and profile.health_score < 95.0:
44 |             return IngestionDecision(strategy=IngestionStrategy.NOVELTY_SIGNATURE, rationale=f'Обнаружено {len(profile.novel_signatures)} новых сигнатур/событий. Обогащение базы знаний RAG новыми паттернами.', chunks_to_generate=len(profile.novel_signatures) + 1, target_time_windows=[], skip_providers=[profile.dominant_noise_provider] if profile.redundancy_ratio_pct > 8

```

```
5 else [], recommended_llm_action='Классифицировать новые сигн  
атуры и обновить локальную базу знаний.')  
45     if profile.redundancy_ratio_pct >= 90.0:  
46         return IngestionDecision(strategy=IngestionStrateg  
y.NOISE_MASKED, rationale=f'Массив на {profile.redundancy_rati  
o_pct}% состоит из однотипного монотонного шума (доминирует {p  
rofile.dominant_noise_provider}). Создается 1 агрегированный д  
айджест с маскированием.', chunks_to_generate=min(5, max(1, pr  
ofile.unique_templates_count)), target_time_windows=[], skip_p  
roviders=[profile.dominant_noise_provider], recommended_llm_ac  
tion='Сжать и зафиксировать статистику фоновых сервисов без ли  
шних токенов.')  
47     return IngestionDecision(strategy=IngestionStrategy.SN  
APSHOT_ONLY, rationale=f'Система полностью стабильна: Health S  
core {profile.health_score}%, 0 сбоев. В RAG сохраняется тольк  
о 1 почасовой снимок здоровья ОС (0 токенов на события).', chu  
nks_to_generate=1, target_time_windows=[], skip_providers=[],  
recommended_llm_action='Система в норме. Вмешательство ИИ не т  
ребуется.')
```

## models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Models
4 # =====
5 # Description:
6 #   Enumeration of standard log severity levels.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.src.models import Log
      Severity
11 #
12 #       service = LogSeverity()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.src
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Enumeration of standard log severity levels."""
24
25 from dataclasses import dataclass, field
26 from enum import StrEnum
27 from typing import Any, Dict, List, Optional
28
29 class LogSeverity(StrEnum):
30     """Enumeration of standard log severity levels."""
31     CRITICAL = 'Critical'
```

```

32     ERROR = 'Error'
33     WARNING = 'Warning'
34     INFORMATION = 'Information'
35     VERBOSE = 'Verbose'
36     DEBUG = 'Debug'
37
38 class IngestionStrategy(StrEnum):
39     """Стратегия обработки и индексации данных, выбранная Deci
    sion Gate."""
40     SNAPSHOT_ONLY = 'snapshot_only'
41     INCIDENT_FOCUSED = 'incident_focused'
42     NOVELTY_SIGNATURE = 'novelty_signature'
43     NOISE_MASKED = 'noise_masked'
44
45 @dataclass
46 class LogEntry:
47     """Нормализованная запись журнала событий Windows."""
48     timestamp: str = ''
49     level: str = LogSeverity.INFORMATION.value
50     source: str = ''
51     provider: str = ''
52     channel: str = ''
53     event_id: int = 0
54     computer: str = ''
55     user: str = ''
56     process: str = ''
57     process_id: int = 0
58     thread_id: int = 0
59     service: str = ''
60     message: str = ''
61     raw_data: str = ''
62     metadata: Dict[str, Any] = field(default_factory=dict)
63
64 @dataclass
65 class TimeBurst:
66     """Временной всплеск частоты событий (Rate Spike)."""
67     time_window: str = ''
68     event_count: int = 0

```

```

69     dominant_level: str = 'Information'
70     dominant_provider: str = ''
71     summary: str = ''
72
73 @dataclass
74 class DataProfileReport:
75     """Аналитический профиль массива логов от Data Researcher
76     (EDA)."""
77     channel: str = ''
78     total_events: int = 0
79     unique_templates_count: int = 0
80     redundancy_ratio_pct: float = 0.0
81     health_score: float = 100.0
82     critical_count: int = 0
83     error_count: int = 0
84     warning_count: int = 0
85     info_count: int = 0
86     dominant_noise_provider: str = ''
87     bursts: List[TimeBurst] = field(default_factory=list)
88     critical_incidents: List[Dict[str, Any]] = field(default_factory=list)
89     novel_signatures: List[Dict[str, Any]] = field(default_factory=list)
90     top_patterns: List[Dict[str, Any]] = field(default_factory=list)
91     generated_at: str = ''
92
93 @dataclass
94 class IngestionDecision:
95     """Решение Decision Gate о том, как обрабатывать и индексировать массив."""
96     strategy: IngestionStrategy = IngestionStrategy.SNAPSHOT_ONLY
97     rationale: str = ''
98     chunks_to_generate: int = 0
99     target_time_windows: List[str] = field(default_factory=list)
100     skip_providers: List[str] = field(default_factory=list)

```

```
100 | recommended_llm_action: str = ''
```

## pipeline.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Pipeline
4 # =====
5 # Description:
6 #   Единый фасад пайплайна: Data Researcher -> Decision Gate -
  > Adaptive RAG.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.src.pipeline import L
  ogIntelligencePipeline
11 #
12 #       service = LogIntelligencePipeline()
13 #
14 # File: pipeline.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.src
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Единый фасад пайплайна: Data Researcher -> Decision Gate ->
  Adaptive RAG."""
24
25 from typing import Any, Dict, List, Optional
26 from .adaptive_rag import AdaptiveLogRAG
27 from .data_researcher import LogDataResearcher
28 from .decision_gate import DecisionGate
29 from .models import DataProfileReport, IngestionDecision, LogE
```

```

ntry
30
31 class LogIntelligencePipeline:
32     """Единый фасад пайплайна: Data Researcher -> Decision Gate -> Adaptive RAG."""
33
34     def __init__(self, storage_dir: Optional[Any]=None) -> None:
35         self.researcher = LogDataResearcher()
36         self.gate = DecisionGate()
37         self.rag = AdaptiveLogRAG(storage_dir=storage_dir)
38
39     def process_events(self, entries: List[LogEntry], channel: str='System') -> Dict[str, Any]:
40         """Выполнить полный цикл аналитики, принятия решения и адаптивной индексации."""
41         profile: DataProfileReport = self.researcher.profile_data(entries, channel=channel)
42         decision: IngestionDecision = self.gate.evaluate(profile)
43         chunks_created = self.rag.ingest(profile, decision)
44         return {'channel': channel, 'profile': {'total_events': profile.total_events, 'unique_templates': profile.unique_templates_count, 'redundancy_ratio_pct': profile.redundancy_ratio_pct, 'health_score': profile.health_score, 'critical_count': profile.critical_count, 'error_count': profile.error_count, 'warning_count': profile.warning_count, 'bursts_count': len(profile.bursts), 'novel_signatures_count': len(profile.novel_signatures)}, 'decision': {'strategy': decision.strategy.value, 'rationale': decision.rationale, 'chunks_generated': chunks_created, 'recommended_llm_action': decision.recommended_llm_action}, 'rag_storage_dir': str(self.rag.storage_dir), 'generated_at': profile.generated_at}
45
46     def search_rag(self, query: str, top_k: int=5, channel: str='') -> List[Dict[str, Any]]:
47         """Поиск по адаптивным документам RAG."""
48         return self.rag.search(query=query, top_k=top_k, channel=channel)

```

```
|el=channel)
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Sr
  c - Init
4 # =====
5 # Description:
6 #   Модуль реализации компонентов подсистемы Windows AI-Breadb
  oard (__init__).
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.log_intelligence.src
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Модуль реализации компонентов подсистемы Windows AI-Breadbo
  ard (__init__)."""
20
21 from .models import DataProfileReport, IngestionDecision, Inge
  stionStrategy, LogEntry, LogSeverity, TimeBurst
22 from .data_researcher import LogDataResearcher
23 from .decision_gate import DecisionGate
24 from .adaptive_rag import AdaptiveLogRAG, get_default_storage_
  dir
25 from .pipeline import LogIntelligencePipeline
26 __all__ = ['DataProfileReport', 'IngestionDecision', 'Ingestio
  nStrategy', 'LogEntry', 'LogSeverity', 'TimeBurst', 'LogDataRe
  searcher', 'DecisionGate', 'AdaptiveLogRAG', 'get_default_stor
```

```
|age_dir', 'LogIntelligencePipeline']
```

## test\_intelligence\_pipeline.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Log_Intelligence Tests - Test Intelligence Pipeline
4 # =====
5 # Description:
6 #   Проверка работы Data Researcher: подсчет Redundancy Ratio, Health Score и детекция всплесков.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.log_intelligence.tests.test_intelligence_pipeline import test_data_researcher_profiling
11 #
12 #       res = test_data_researcher_profiling()
13 #
14 # File: test_intelligence_pipeline.py
15 # Project: ai-breadboard
16 # Package: apps.windows.log_intelligence.tests
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 05:04:00
20 # =====
21
22 """Проверка работы Data Researcher: подсчет Redundancy Ratio, Health Score и детекция всплесков."""
23
24 from pathlib import Path
25 import pytest
26 from apps.windows.log_intelligence.src.models import DataProfileReport, IngestionDecision, IngestionStrategy, LogEntry, LogSeverity
27 from apps.windows.log_intelligence.src.data_researcher import
```

```

LogDataResearcher
28 from apps.windows.log_intelligence.src.decision_gate import DecisionGate
29 from apps.windows.log_intelligence.src.adaptive_rag import AdaptiveLogRAG, get_default_storage_dir
30 from apps.windows.log_intelligence.src.pipeline import LogIntelligencePipeline
31
32 def test_data_researcher_profiling() -> None:
33     """Проверка работы Data Researcher: подсчет Redundancy Ratio, Health Score и детекция всплесков."""
34     researcher = LogDataResearcher()
35     entries = [LogEntry(timestamp='2026-09-16 15:00:01', level='Information', provider='Service', event_id=7036, message='Service started'), LogEntry(timestamp='2026-09-16 15:00:02', level='Information', provider='Service', event_id=7036, message='Service started'), LogEntry(timestamp='2026-09-16 15:00:03', level='Information', provider='Service', event_id=7036, message='Service started'), LogEntry(timestamp='2026-09-16 15:05:00', level='Error', provider='Kernel-Power', event_id=41, message='Reboot without shutdown'), LogEntry(timestamp='2026-09-16 15:05:01', level='Warning', provider='rt640x64', event_id=1, message='Network disconnected')]
36     profile = researcher.profile_data(entries, channel='System')
37     assert profile.total_events == 5
38     assert profile.unique_templates_count < 5
39     assert profile.redundancy_ratio_pct > 0.0
40     assert profile.error_count == 1
41     assert profile.warning_count == 1
42     assert profile.health_score < 100.0
43
44 def test_decision_gate_strategies() -> None:
45     """Проверка выбора стратегий в Decision Gate."""
46     gate = DecisionGate()
47     err_profile = DataProfileReport(channel='System', total_events=100, unique_templates_count=20, redundancy_ratio_pct=80.0, health_score=65.0, error_count=2)

```

```

48     dec1 = gate.evaluate(err_profile)
49     assert dec1.strategy == IngestionStrategy.INCIDENT_FOCUSED
50     healthy_profile = DataProfileReport(channel='System', total_events=100, unique_templates_count=15, redundancy_ratio_pct=85.0, health_score=100.0, error_count=0, warning_count=0)
51     dec2 = gate.evaluate(healthy_profile)
52     assert dec2.strategy == IngestionStrategy.SNAPSHOT_ONLY
53     noisy_profile = DataProfileReport(channel='System', total_events=500, unique_templates_count=5, redundancy_ratio_pct=99.0, health_score=98.0, dominant_noise_provider='Kernel-PnP')
54     dec3 = gate.evaluate(noisy_profile)
55     assert dec3.strategy == IngestionStrategy.NOISE_MASKED
56
57 def test_adaptive_rag_pipeline(tmp_path: Path) -> None:
58     """Проверка полного цикла LogIntelligencePipeline с сохранением и поиском."""
59     pipeline = LogIntelligencePipeline(storage_dir=tmp_path / 'rag_storage')
60     entries = [LogEntry(timestamp='2026-09-16 16:30:00', level='Warning', provider='rt640x64', event_id=1, message='Realtek network adapter disconnected'), LogEntry(timestamp='2026-09-16 16:30:05', level='Error', provider='BITS', event_id=16393, message='BITS gateway communication failure 0x80040500')]
61     res = pipeline.process_events(entries, channel='System')
62     assert res['profile']['total_events'] == 2
63     assert res['decision']['chunks_generated'] > 0
64     search_res = pipeline.search_rag('почему отключился сетевой адаптер?')
65     assert len(search_res) > 0
66     assert 'Realtek' in search_res[0]['text'] or 'network' in search_res[0]['text'].lower() or 'сбо' in search_res[0]['text'].lower()
67
68 def test_adaptive_rag_wikillm_storage(tmp_path: Path) -> None:
69     """Проверка сохранения сущностей телеметрии в SQLite FTS5 хранилище WikiStorage."""
70     rag = AdaptiveLogRAG(storage_dir=tmp_path / 'rag_wiki')
71     gate = DecisionGate()

```

```

72     researcher = LogDataResearcher()
73     entries = [
74         LogEntry(timestamp='2026-09-16 17:00:00', level='Error
', provider='DistributedCOM', event_id=10016, message='The app
lication-specific permission settings do not grant Local Activ
ation permission for the COM Server application'),
75         LogEntry(timestamp='2026-09-16 17:00:05', level='Criti
cal', provider='Kernel-Power', event_id=41, message='The syste
m has rebooted without cleanly shutting down first'),
76     ]
77     profile = researcher.profile_data(entries, channel='System
')
78     decision = gate.evaluate(profile)
79     chunks_count = rag.ingest(profile, decision)
80     assert chunks_count > 0
81
82     # Проверка поиска через SQLite FTS5 в WikiStorage
83     fts_results = rag.wiki_storage.search_fts('DistributedCOM'
)
84     assert len(fts_results) > 0
85     canonical_keys = [e.canonical_key for e in fts_results]
86     assert any('DistributedCOM' in k for k in canonical_keys)
87
88     # Проверка общего поиска
89     search_results = rag.search('DistributedCOM permission')
90     assert len(search_results) > 0
91
92 def test_wevtapi_channel_enumeration() -> None:
93     """Проверка нативного перечисления каналов через WevtAPI."
    """
94     from apps.windows.telemetry.win32_ffi.wevtapi import WevtA
    PI
95     api = WevtAPI()
96     channels = api.enumerate_channels()
97     assert len(channels) >= 10
98     names = [c.channel_name.lower() for c in channels]
99     assert 'system' in names
100    assert 'application' in names

```

```
101 |
102 | def test_log_discovery_engine_sources() -> None:
103 |     """Проверка обнаружения всех источников логов в системе че
    |     рез LogDiscoveryEngine."""
104 |     from apps.windows.core.audits.log_discovery_engine import
    |     LogDiscoveryEngine
105 |     engine = LogDiscoveryEngine()
106 |     sources = engine.discover_all_sources()
107 |     assert len(sources) > 50
108 |     categories = {s.category for s in sources}
109 |     assert 'Windows Event Log' in categories
110 |
```